

EIN092B: Laboratorio de Técnicas de Visualización para Clasificación y Predicción

Guía de Actividades Sesión 9

10 de octubre de 2026

1. Requisitos de entrada.

- Haber **entendido** y **realizado** las actividades de todas las sesiones anteriores.
- Haber revisado los conceptos asociados a las técnicas de aprendizaje supervisado estudiadas en el curso previo de ciencia de datos, tales como regresión logística, *K-nearest neighbor*, máquinas de soporte vectorial y regresión lineal. Adicionalmente, haber revisado técnicas de validación cruzada como *k-fold cross validation*.
- Haber estudiado la codificación de variables categóricas en *one-hot vectors*.

2. Objetivos

- Explorar técnicas de visualización para resultados de modelos de clasificación y regresión.
- Experimentar con distintas métricas de evaluación y gráficos para visualizar errores en tareas de clasificación o regresión.

3. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio por lo menos hasta la Sección 5.1.3. La actividad propuesta en la Sección 5.1.4 puede ser entregada sin validación durante la sesión.

3.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.

- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que **TODOS** los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 3.2.
- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %).**
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

3.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre los códigos propuestos por el grupo. La segunda parte constará de una verificación de sus códigos y respuestas, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se le indican a continuación:

1. Debe crear una carpeta con el nombre *GXX_Sesion9*, donde *XX* el número de grupo. Por ejemplo, la carpeta del grupo 5 se llamaría *G5_Sesion9*. El número de su grupo se definió en la sesión anterior.
2. Dentro de esta carpeta, debe incluirse un archivo de Jupyter Notebook que contenga el desarrollo de las actividades y las respuestas correspondientes.

Recordar que **será responsabilidad del grupo entregar correctamente los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 11 de octubre**. Sin embargo, solo se evaluarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no se considerará en la revisión.

4. Técnicas de evaluación e interpretación para algoritmos de clasificación y regresión

En esta sección se presenta una breve introducción a las técnicas más comunes para evaluar modelos en problemas de regresión y clasificación, y para interpretar sus decisiones. Se discutirán métricas y gráficos fundamentales como la matriz de confusión y la curva ROC para problemas de clasificación, los gráficos de diagnóstico para modelos de regresión y, finalmente, una introducción a la explicabilidad de modelos mediante valores SHAP. El énfasis del laboratorio está en la visualización e interpretación de los resultados, por lo que el entrenamiento de los modelos se entrega ampliamente avanzado.

4.1. Problemas de clasificación

En distintos ámbitos científicos e ingenieriles, para evaluar la eficacia de un algoritmo se suele recurrir a información predefinida que representa el resultado esperado para la resolución de cierto problema. Esta información, conocida como *groundtruth*, se utiliza como referencia al comparar el rendimiento de un algoritmo.

El éxito o fallo de un algoritmo suele evaluarse mediante métricas que cuantifican su rendimiento en la detección de una situación o evento, en comparación con el *groundtruth*. Para esto, existen cuatro categorías generales:

- **Verdadero Positivo (TP):** Ocurre cuando el algoritmo detecta una situación real, es decir, algo que está presente tanto en la *groundtruth* como en los resultados del algoritmo.
- **Falso Negativo (FN):** Corresponde a una situación real que el algoritmo no detecta. Para un problema de clasificación, es el caso en el que el algoritmo no consigue clasificar correctamente según la etiqueta del *groundtruth*.
- **Falso Positivo (FP):** El algoritmo detecta una situación irreal (existe sólo en los resultados del algoritmo).
- **Verdadero Negativo (TN):** Elemento que no aparece ni en el *groundtruth* ni en los resultados del algoritmo.

Una matriz de confusión es una forma conveniente de mostrar y visualizar la información contenida en estas categorías. Permite determinar fácilmente cuáles observaciones fueron etiquetadas incorrectamente y cuáles correctamente. Además, teniendo el número total de observaciones, se pueden obtener estadísticas y la tasa de error global.

A partir de TN, TP, FN, FP se obtienen otras métricas¹:

- **Precisión:** Mide la proporción de verdaderos positivos sobre todos los casos positivos detectados por el algoritmo:

$$\text{Precisión} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (1)$$

¹Existen otras métricas como F1-score, que corresponde a la media armónica entre la precisión y la sensibilidad.

Esta métrica es esencial para entender la capacidad del modelo para evitar identificar erróneamente una señal negativa como positiva.

- **Sensibilidad:** Se define como la proporción de verdaderos positivos sobre todas las situaciones reales:

$$\text{Sensibilidad} = \frac{TP}{TP + FN} \quad (2)$$

Esta métrica refleja la capacidad del modelo para identificar todas las señales positivas en el conjunto de datos.

- **Especificidad:** Mide la proporción de verdaderos negativos sobre todos los casos negativos:

$$\text{Especificidad} = \frac{TN}{FP + TN} \quad (3)$$

Esta métrica cuantifica el rendimiento del algoritmo para evitar detectar erróneamente situaciones irrelevantes.

- **Exactitud o *accuracy*:** Mide el número total de predicciones correctas sobre el total de predicciones realizadas:

$$\text{Exactitud} = \frac{TP + TN}{TP + FP + TN + FN} \quad (4)$$

Cada una de estas métricas ofrece una perspectiva diferente sobre la capacidad de un algoritmo para realizar predicciones precisas y útiles, proporcionando así una base sólida para evaluarlo frente al *groundtruth*. Es importante notar que las distintas métricas penalizan errores distintos: en un contexto clínico, por ejemplo, un falso negativo (no detectar a un paciente enfermo) no tiene el mismo costo que un falso positivo.

4.2. Curva ROC

La curva ROC es una herramienta utilizada principalmente para evaluar el rendimiento de modelos de clasificación binaria. La curva ROC es un gráfico popular para mostrar simultáneamente dos tipos de errores para todos los umbrales posibles. El nombre ROC es histórico y procede de la teoría de las comunicaciones. Es un acrónimo de *receiver operating characteristics* (características operativas del receptor) [3].

La Figura 1 muestra la curva ROC de un clasificador en los datos de entrenamiento. Los dos errores que se pueden visualizar son la *sensibilidad* y uno menos la *especificidad*. El rendimiento global de un clasificador, sumado sobre todos los umbrales posibles, viene dado por el *área bajo la curva* (AUC). En el caso de un clasificador ideal, la curva ROC deberá estar cerca de la esquina superior izquierda, por lo que cuanto mayor sea el AUC, mejor será el clasificador. Para los datos en la Figura 1, el AUC es de 0.95, que se acerca al máximo de 1.0, por lo que se consideraría muy bueno. Un clasificador que no es mejor que el azar tiene un AUC de 0.5; por lo tanto, se espera que un buen clasificador tenga un AUC significativamente mayor a 0.5 cuando se evalúa en un conjunto de prueba independiente, no utilizado en el entrenamiento del modelo.

Para un modelo que produce probabilidades para cada clase, es posible ajustar un umbral de decisión. Al variar dicho umbral, cambian la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR). Estas corresponden a la *sensibilidad* y a una menor *especificidad* del clasificador,

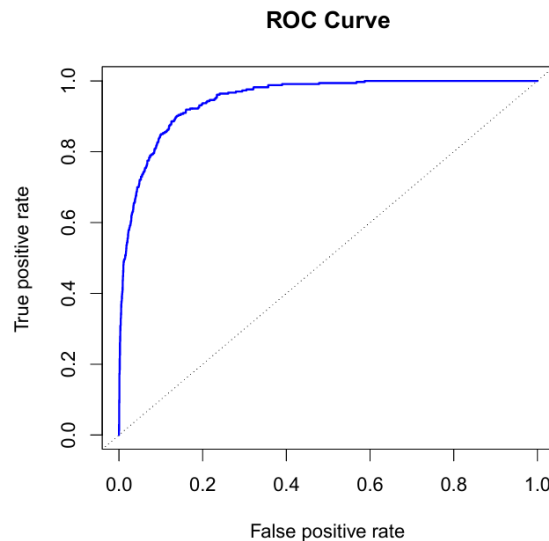


Figura 1: Ejemplo de curva ROC. Esta curva muestra cómo varían dos tipos de error al ajustar el umbral de clasificación. La tasa de verdaderos positivos (*sensibilidad*) representa la fracción de casos correctamente identificados con un umbral dado, mientras que la tasa de falsos positivos ($1 - \textit{especificidad}$) indica la fracción de casos negativos clasificados incorrectamente como positivos con el mismo umbral.

respectivamente. La curva ROC ideal abraza la esquina superior izquierda, lo que indica una alta tasa de verdaderos positivos y una baja tasa de falsos positivos.

Las curvas ROC son útiles para comparar distintos clasificadores, ya que consideran todos los umbrales posibles. El uso de la curva ROC tiene algunas restricciones: se requieren las probabilidades (o algún puntaje continuo) de que cada muestra pertenezca a la clase positiva. Esto es aplicable a algoritmos como las redes neuronales artificiales o la regresión logística, entre otros.

4.3. Problemas de regresión

Para evaluar algoritmos que resuelven problemas de regresión se utilizan métricas de error como *mean absolute error* (MAE), *mean squared error* (MSE), *root mean squared error* (RMSE) o el coeficiente de determinación (R^2). Comúnmente, estas métricas se representan en *box plots* o *scatter plots*, y cuando se obtienen múltiples valores para las métricas (como en *cross-validation*) es recomendable mostrar la variación entre *folds* mediante un *box plot* o una barra de desviación estándar.

Una métrica resumida puede ocultar patrones importantes en los errores, por lo que se recomienda complementarla con gráficos de diagnóstico:

- **Valor real vs. valor predicho:** Si el modelo fuera perfecto, todos los puntos caerían sobre la diagonal $y = x$. La dispersión alrededor de la diagonal permite apreciar la magnitud del error y si el modelo subestima o sobreestima sistemáticamente ciertos rangos.
- **Residuos vs. valor predicho:** Los residuos ($y - \hat{y}$) deberían distribuirse de forma aleatoria alrededor de cero. Patrones como curvaturas sugieren relaciones no capturadas por el modelo,

y una dispersión que crece con el valor predicho (forma de embudo) sugiere varianza de error no constante.

- **Histograma de residuos:** Permite verificar si los errores están centrados en cero y detectar valores atípicos.

Gráficos como la curva ROC no son aplicables directamente a problemas de regresión, debido a que estos no producen valores de decisión binarios ni probabilidades. Sin embargo, un problema de regresión puede transformarse en uno de clasificación binaria (por ejemplo, seleccionando un umbral para categorizar un valor continuo en clase 0 o 1), y en ese caso sí se podría construir una curva ROC.

4.4. Explicabilidad de modelos con SHAP

Un modelo con buen desempeño no necesariamente es un modelo comprensible. La explicabilidad (o interpretabilidad) busca responder por qué un modelo entrega cierta predicción, lo cual es especialmente relevante en dominios como la salud, donde las decisiones deben poder justificarse y contrastarse con el conocimiento experto.

Los valores SHAP (*SHapley Additive exPlanations*) se basan en los valores de Shapley de la teoría de juegos cooperativos: la predicción del modelo se “reparte” entre las variables de entrada según su contribución [2]. Para una variable i dentro del conjunto de variables F , su valor de Shapley es el promedio ponderado de su contribución marginal sobre todos los subconjuntos posibles de las demás variables:

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (5)$$

Esta formulación cumple una propiedad de aditividad: la predicción para un paciente es igual a un valor base ϕ_0 (la predicción promedio sobre un conjunto de datos de referencia o *background*) más la suma de las contribuciones de cada variable:

$$f(x) = \phi_0 + \sum_{i=1}^{|F|} \phi_i \quad (6)$$

Algunos conceptos clave para interpretar los resultados:

- **Explicaciones locales y globales:** Los valores SHAP explican predicciones individuales (gráfico *waterfall*), y al agregarlos sobre muchos pacientes permiten una visión global de qué variables son más influyentes (gráficos *bar* y *beeswarm*).
- **Explicadores específicos y agnósticos al modelo:** Para modelos lineales existe un cálculo eficiente y exacto (`LinearExplainer`); para modelos arbitrarios, como KNN, se aproximan los valores mediante muestreo y permutaciones de variables, lo cual es más costoso y depende del conjunto de referencia elegido.

- **Unidades de la explicación:** Los valores SHAP se expresan en las unidades de la salida explicada. Para la regresión logística suele ser el *log-odds*, mientras que para KNN, explicado a través de `predict_proba`, es probabilidad. Por ello, las magnitudes entre modelos no son directamente comparables; sí lo son, con cautela, los rankings de importancia.
- **Limitaciones:** SHAP explica el comportamiento *del modelo*, no la realidad. Una variable con alto valor SHAP indica que el modelo la utiliza, no que sea causa de la enfermedad. Además, con variables correlacionadas, la contribución puede repartirse entre ellas de formas no evidentes.

5. Actividades

En las siguientes subsecciones se describen las actividades a realizar. En cada actividad encontrará indicaciones generales sobre el conjunto de datos a emplear y los requerimientos. En este laboratorio se emplean técnicas clásicas para evaluar algoritmos de clasificación y regresión, y técnicas de explicabilidad para interpretarlos.

El código de carga, preparación de los datos y entrenamiento de los modelos se entrega ya implementado, de modo que el trabajo se concentre en las **visualizaciones y en su interpretación**. Todos los gráficos entregados deben incluir título, etiquetas de ejes y leyenda cuando corresponda, y cada uno debe acompañarse de una breve interpretación escrita.

5.1. Aplicación de aprendizaje automático

En esta sección se emplea el dataset público de enfermedades cardíacas [1]. El propósito es aplicar técnicas basadas en datos para abordar un problema relacionado con la detección de enfermedades cardíacas. El dataset *Heart disease* está disponible en *Kaggle*², donde puede acceder además a una descripción de las variables. En esta actividad se trabaja con el archivo `heart.dat`.

5.1.1. Carga y preparación de los datos

El conjunto de datos contiene variables categóricas. Para conservar su representación original y facilitar el análisis e interpretación posteriores, se provee el siguiente código de referencia de carga:

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 headers = ['age', 'sex', 'chest_pain', 'blood_p', 'serum', 'blood_s', 'electro',
7           'max_heart', 'angina', 'oldpeak', 'slope', 'vessel', 'thal', 'disease']
8 df = pd.read_csv("heart.dat", header=None, names=headers, sep=r"\s+")
9 df['disease'] = df['disease'] - 1 # 0: ausencia, 1: presencia
10
11 # Recuperar la representacion original de las variables categoricas
12 category_maps = {

```

²Heart disease dataset en Kaggle.

```

13     'sex': {0: 'female', 1: 'male'},
14     'chest_pain': {1: 'typical angina', 2: 'atypical angina',
15                   3: 'non-anginal pain', 4: 'asymptomatic'},
16     'blood_s': {0: 'lower than 120mg/dl', 1: 'greater than 120mg/dl'},
17     'electro': {0: 'normal', 1: 'ST-T wave abnormality',
18                2: 'left ventricular hypertrophy'},
19     'angina': {0: 'no', 1: 'yes'},
20     'slope': {1: 'upsloping', 2: 'flat', 3: 'downsloping'},
21     'thal': {3: 'normal', 6: 'fixed defect', 7: 'reversible defect'},
22 }
23 for col, mapping in category_maps.items():
24     df[col] = df[col].astype(int).map(mapping)
25
26 df.head()

```

Antes de utilizar los datos en un modelo, las variables categóricas deben codificarse como *one-hot vectors*, indicando con un 1 la presencia del atributo en cuestión. Por ejemplo, si un dato recolectado para un paciente incluye el atributo 'angina: yes', este se codificará como [0,1], mientras que si incluye el atributo 'angina: no', se codificará como [1,0].

```

1 df = pd.get_dummies(df, dtype=float) # one-hot de las variables categoricas
2 df.head()

```

Finalmente, se separan y estandarizan los datos. Se reserva el 30% de los datos para el conjunto de prueba (*test*), manteniendo la proporción de pacientes sanos y enfermos, y la estandarización se calcula únicamente con el conjunto de entrenamiento. Se preparan dos tareas a partir de la misma partición: regresión de la presión sanguínea (*blood_p*) y clasificación de la enfermedad (*disease*).

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 y_cls = df['disease'] # 0: ausencia, 1: presencia
5 X = df.drop(columns='disease') # incluye blood_p
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y_cls, test_size=0.3, stratify=y_cls, random_state=42)
9
10 # regresion: se excluye el target (presi n sangu nea) de las entradas
11 yreg_train, yreg_test = X_train['blood_p'], X_test['blood_p']
12 Xr_train = X_train.drop(columns='blood_p')
13 Xr_test = X_test.drop(columns='blood_p')
14
15 def standardize(train, test):
16     sc = StandardScaler().fit(train) # estadisticos solo con entrenamiento
17     f = lambda d: pd.DataFrame(sc.transform(d), columns=d.columns, index=d.index)
18     return f(train), f(test)
19
20 X_train_std, X_test_std = standardize(X_train, X_test) # clasificacion
21 Xr_train_std, Xr_test_std = standardize(Xr_train, Xr_test) # regresion

```

5.1.2. Predicción de la presión sanguínea (40 puntos)

El objetivo de esta sección es predecir el valor de la presión sanguínea en función de las demás variables, para luego cuantificar la diferencia entre el valor real y el valor predicho. Como referen-

cia, valores anormales de presión sanguínea pueden ser un indicador de problemas cardíacos; por lo tanto, la diferencia entre lo observado y lo esperado podría contener información sobre el estado del paciente.

Actividades a realizar:

- **Exploración visual.** Realice un máximo de cuatro gráficos para inspeccionar el conjunto de datos. Deben incluirse al menos: (i) un gráfico que compare la distribución de `blood_p` entre pacientes sanos y enfermos (por ejemplo, *boxplot* o *violin plot*); (ii) un mapa de calor de correlaciones entre las variables numéricas; y (iii) un gráfico de su elección, justificando por qué lo considera relevante. Interprete adecuadamente los resultados.
- **Selección del modelo.** Elija uno de los modelos disponibles modificando la variable `MODEL_NAME` en el código entregado, y justifique su elección según su conocimiento previo sobre el modelo. Los hiperparámetros ya están fijados, por lo que no se requiere una búsqueda de hiperparámetros.

```

1 from sklearn.linear_model import LinearRegression, Ridge
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.neighbors import KNeighborsRegressor
4 from sklearn.metrics import mean_absolute_error, mean_squared_error
5
6 REG_MODELS = {
7     'linear': LinearRegression(),
8     'ridge': Ridge(alpha=10.0),
9     'random_forest': RandomForestRegressor(n_estimators=200, max_depth=5,
10     random_state=42),
11     'knn': KNeighborsRegressor(n_neighbors=10),
12 }
13 MODEL_NAME = 'ridge' # elija: 'linear', 'ridge', 'random_forest' o 'knn'
14
15 model = REG_MODELS[MODEL_NAME]
16 model.fit(Xr_train_std, yreg_train)
17 pred_train = model.predict(Xr_train_std)
18 pred_test = model.predict(Xr_test_std)
19
20 for name, y_true, y_pred in [('train', yreg_train, pred_train),
21                             ('test', yreg_test, pred_test)]:
22     print(f"{name}: MAE = {mean_absolute_error(y_true, y_pred):.2f} | "
23           f"MSE = {mean_squared_error(y_true, y_pred):.2f}")

```

- **Diagnóstico visual del modelo.** A partir de las predicciones anteriores, genere: (i) un gráfico de dispersión de valor real vs. valor predicho, con la diagonal $y = x$ como referencia, en una distribución de 1×2 ; y (ii) un gráfico de residuos vs. valor predicho y un histograma de los residuos, ambos para el conjunto de prueba. Discuta los resultados: ¿El modelo subestima o sobreestima ciertos rangos de presión? ¿Hay patrones en los residuos? ¿Existen diferencias importantes entre entrenamiento y prueba?
- **Validación cruzada.** Se entrega el código para evaluar el modelo mediante *cross-validation* con $K = 5$ folds, donde los pesos del modelo se reajustan en cada *fold* y la estandarización se calcula dentro de cada partición. Realice un *boxplot* de los valores de MAE y MSE obtenidos en cada *fold* y reporte ambas métricas como $\text{MEAN} \pm \text{STD}$. Compare estos valores con los obtenidos en el conjunto de prueba y discuta la variabilidad observada entre *folds*.

```

1 from sklearn.base import clone
2 from sklearn.model_selection import KFold, cross_validate, cross_val_predict
3 from sklearn.pipeline import make_pipeline
4
5 kf = KFold(n_splits=5, shuffle=True, random_state=42)
6 pipe = make_pipeline(StandardScaler(), clone(model))
7
8 cv = cross_validate(pipe, Xr_train, yreg_train, cv=kf,
9                     scoring={'MAE': 'neg_mean_absolute_error',
10                             'MSE': 'neg_mean_squared_error'})
11 df_cv = pd.DataFrame({'MAE': -cv['test_MAE'], 'MSE': -cv['test_MSE']})
12 print(df_cv.round(2))
13 print(df_cv.agg(['mean', 'std']).round(2))

```

- **Residuos y enfermedad.** Para explorar la hipótesis de que un comportamiento anormal de la presión sanguínea es un indicio de enfermedad, se entrega el código que calcula, para todos los pacientes, una predicción fuera de muestra (obtenida con *cross-validation*, de modo que ningún paciente es predicho por un modelo que lo haya visto durante su entrenamiento). Genere un *boxplot* o *violin plot* de los residuos agrupados por estado del paciente (sano/enfermo). ¿Respalda este gráfico la hipótesis? ¿Qué tan grande es la diferencia respecto a la variabilidad total de los residuos?

```

1 Xr_all = X.drop(columns='blood_p')
2 yr_all = X['blood_p']
3 pred_cv = cross_val_predict(pipe, Xr_all, yr_all, cv=kf)
4
5 df_res = pd.DataFrame({
6     'real': yr_all,
7     'pred': pred_cv,
8     'residual': yr_all - pred_cv,
9     'status': y_cls.map({0: 'sano', 1: 'enfermo'}),
10 })
11 df_res.head()

```

5.1.3. Predicción de la enfermedad cardíaca (40 puntos)

En esta sección nos enfocaremos en la predicción de la enfermedad cardíaca, un problema de clasificación binaria. Se utilizan dos algoritmos de clasificación, *K-nearest neighbors* (KNN) y *logistic regression* (LR), con hiperparámetros ya definidos. Se entrega el código que los entrena sobre los datos estandarizados y calcula sus métricas en el conjunto de prueba.

```

1 from sklearn.linear_model import LogisticRegression
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.metrics import (accuracy_score, precision_score,
4                             recall_score, roc_auc_score)
5
6 CLF_MODELS = {
7     'KNN': KNeighborsClassifier(n_neighbors=7),
8     'LR': LogisticRegression(max_iter=1000),
9 }
10
11 results = []
12 for name, clf in CLF_MODELS.items():

```

```

13 clf.fit(X_train_std, y_train)
14 pred = clf.predict(X_test_std)
15 proba = clf.predict_proba(X_test_std)[: , 1]
16 results.append({'model': name,
17                 'accuracy': accuracy_score(y_test, pred),
18                 'precision': precision_score(y_test, pred),
19                 'recall': recall_score(y_test, pred),
20                 'auc': roc_auc_score(y_test, proba)})
21 pd.DataFrame(results).round(3)

```

Actividades a realizar:

- **Curva ROC y matriz de confusión.** Genere un único gráfico con las curvas ROC de ambos modelos (indicando el AUC en la leyenda y la diagonal del azar como referencia) y las matrices de confusión de ambos modelos en una distribución de 1×2 . Puede apoyarse en `RocCurveDisplay` y `ConfusionMatrixDisplay`. Si bien KNN no es un modelo probabilístico en sentido estricto, `scikit-learn` permite obtener una probabilidad a partir de la proporción de vecinos de cada clase mediante `predict_proba`. Discuta los resultados: ¿qué forma tiene la curva ROC de KNN y por qué difiere de la de LR?
- **Elección del umbral.** En un contexto clínico, no detectar a un paciente enfermo (falso negativo) puede ser más costoso que una falsa alarma (falso positivo). Seleccione, para el modelo de su preferencia, un umbral de decisión que garantice una sensibilidad de al menos 0.90 en el conjunto de prueba. Marque este punto de operación sobre la curva ROC, genere la matriz de confusión y discuta qué se gana y qué se pierde respecto al umbral por defecto (0.5).
- **Proyección con PCA.** Con el fin de obtener una explicación visual de la clasificación, proyecte los datos estandarizados de entrenamiento y prueba en 2 dimensiones mediante PCA. Genere un gráfico de dispersión de los datos de prueba en una distribución de 1×3 : coloreados por la clase real, por la predicción de KNN y por la predicción de LR (modelos de la tabla anterior). Comente el comportamiento: ¿dónde se concentran los errores?
- **Fronteras de decisión.** Se entrega el código que entrena ambos clasificadores directamente sobre las 2 componentes principales y dibuja sus fronteras de decisión. Interprete el resultado: ¿qué diferencias observa entre la frontera de un modelo lineal y la de KNN? Considere que estos modelos se reentrenaron sobre la proyección, por lo que no son idénticos a los utilizados en la tabla de métricas.

```

1 from sklearn.base import clone
2 from sklearn.decomposition import PCA
3 from sklearn.inspection import DecisionBoundaryDisplay
4
5 pca2 = PCA(n_components=2).fit(X_train_std)
6 Z_train, Z_test = pca2.transform(X_train_std), pca2.transform(X_test_std)
7
8 fig, axes = plt.subplots(1, 2, figsize=(11, 4))
9 for ax, (name, clf) in zip(axes, CLF_MODELS.items()):
10     m = clone(clf).fit(Z_train, y_train)
11     DecisionBoundaryDisplay.from_estimator(m, Z_train, response_method='
12                                     predict',
13                                     alpha=0.3, ax=ax)
14     ax.scatter(Z_test[:, 0], Z_test[:, 1], c=y_test, edgecolor='k')

```

```

14 ax.set_title(name)
15 ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
16 plt.show()

```

- **Efecto de la dimensionalidad.** Se entrega el código que entrena los modelos con una representación reducida por PCA para cada número de componentes k y almacena la varianza explicada acumulada, la exactitud y el AUC en el conjunto de prueba. Genere: (i) un gráfico de la varianza explicada acumulada en función de k ; y (ii) un gráfico del AUC (o exactitud) de cada modelo en función de k , incluyendo como referencia una línea horizontal con el desempeño obtenido con la representación original. Responda: ¿Cuántas componentes se necesitan para alcanzar un desempeño similar al de la representación original? ¿Es esperable este fenómeno? ¿Podría la representación reducida mejorar la predicción? ¿Qué relación observa entre la varianza explicada y el desempeño predictivo?

```

1 rows = []
2 for k in range(1, X_train_std.shape[1] + 1):
3     pca = PCA(n_components=k).fit(X_train_std)
4     Z_tr, Z_te = pca.transform(X_train_std), pca.transform(X_test_std)
5     for name, clf in CLF_MODELS.items():
6         m = clone(clf).fit(Z_tr, y_train)
7         rows.append({'k': k, 'model': name,
8                     'var_acum': pca.explained_variance_ratio_.sum(),
9                     'accuracy': accuracy_score(y_test, m.predict(Z_te)),
10                    'auc': roc_auc_score(y_test, m.predict_proba(Z_te)[: ,
11                                     1]) })
11 df_pca = pd.DataFrame(rows)
12 df_pca.head()

```

5.1.4. Explicabilidad de los modelos (20 puntos)

En esta actividad se busca analizar y explicar el comportamiento de los modelos de clasificación entrenados previamente (KNN y LR), utilizando técnicas de interpretabilidad basadas en SHAP (*SHapley Additive exPlanations*). El objetivo es identificar qué variables tienen mayor impacto en la decisión de cada modelo, comparar ambos clasificadores y discutir la coherencia de los resultados con el problema planteado. Para más información, consulte la documentación oficial de SHAP³.

Se entrega el código que calcula los valores SHAP para ambos modelos sobre el conjunto de prueba. Para LR se utiliza un explicador lineal exacto; para KNN, un explicador agnóstico al modelo basado en un subconjunto de referencia de los datos de entrenamiento. En ambos casos, se explica la clase 1 (presencia de enfermedad).

```

1 import shap
2
3 lr, knn = CLF_MODELS['LR'], CLF_MODELS['KNN']
4
5 # --- Regresión logística (modelo lineal): explicación en log-odds ---
6 explainer_lr = shap.LinearExplainer(lr, X_train_std)
7 shap_lr = explainer_lr(X_test_std)
8
9 # --- KNN (modelo no lineal): explicación en probabilidad ---

```

³<https://shap.readthedocs.io/>

```

10 background = shap.sample(X_train_std, 100, random_state=0)
11 explainer_knn = shap.Explainer(knn.predict_proba, background)
12 shap_knn = explainer_knn(X_test_std)[: , : , 1] # clase 1: enfermedad
13
14 # --- Selección de pacientes para el análisis local ---
15 pred_lr = lr.predict(X_test_std)
16 idx_ok = np.where((y_test.values == 1) & (pred_lr == 1))[0][0] # enfermo
    detectado
17 idx_err = np.where(y_test.values != pred_lr)[0][0] # error de LR

```

Actividades a realizar:

- **Importancia global.** Genere los gráficos *beeswarm* y de barras (`shap.plots.beeswarm` y `shap.plots.bar`) para ambos modelos. Interprete cómo el valor de cada variable (alto o bajo) empuja la predicción hacia la presencia o ausencia de enfermedad. Compare las cinco variables más influyentes de cada modelo: ¿coinciden? Recuerde que las magnitudes de SHAP de ambos modelos están en unidades distintas (*log-odds* vs. probabilidad), por lo que debe comparar rankings y no valores absolutos.

```

1 # Beeswarm: un punto por paciente y por variable
2 shap.plots.beeswarm(shap_lr, max_display=15, show=False)
3 plt.title('Beeswarm - Regresión logística')
4 plt.tight_layout()
5 plt.show()
6
7 # Barras: importancia global (promedio del valor absoluto de SHAP)
8 shap.plots.bar(shap_lr, max_display=15, show=False)
9 plt.title('Importancia global - Regresión logística')
10 plt.tight_layout()
11 plt.show()
12
13 # Ranking numerico de las 5 variables mas influyentes
14 imp_lr = pd.Series(np.abs(shap_lr.values).mean(axis=0), index=X_test_std.
    columns)
15 print(imp_lr.sort_values(ascending=False).head(5))

```

- **Explicación local.** Genere un gráfico *waterfall* (`shap.plots.waterfall`) para dos pacientes: uno correctamente identificado como enfermo (`idx_ok`) y uno donde LR comete un error (`idx_err`). Explique qué variables llevaron al modelo a su predicción en cada caso y, para el paciente mal clasificado, proponga una hipótesis de por qué el modelo se equivocó. ¿Cambia la explicación si utiliza el mismo paciente en KNN?
- **Efecto de una variable.** Seleccione una variable numérica (por ejemplo, `max_heart`, `oldpeak` o `age`) y genere el gráfico de dependencia `shap.plots.scatter` para LR y para KNN. ¿Qué forma tiene la relación entre el valor de la variable y su contribución a la predicción en cada modelo? ¿Cómo se relaciona esto con que LR sea lineal y KNN no?

Referencias

- [1] Andras Janosi et al. "Heart disease. uci machine learning repository". En: *UCI Machine Learning Repository* (1988).

- [2] Scott M Lundberg y Su-In Lee. “A unified approach to interpreting model predictions”. En: *Advances in neural information processing systems* 30 (2017).
- [3] Gareth James et al. *An introduction to statistical learning: With applications in python*. Springer Nature, 2023.